

**Перечень вопросов к экзамену по дисциплине
«Алгоритмические языки»**

2 семестр

1. Классы в языке Си++: объявление, поля и функции (методы) классов. Определение функций внутри класса и за пределами класса, примеры.
2. Статусы доступа компонент класса. Операции для доступа к компонентам класса, примеры.
3. Конструкторы и деструктор класса, конструктор копирования и конструктор перемещения, назначение, пример.
4. Статические компоненты класса, назначение, пример.
5. Указатель this, примеры использования.
6. Указатели на компоненты класса (на функции и поля класса), назначение, пример.
7. Дружественные функции и классы, назначение, пример.
8. Шаблоны классов в Си++. Примеры использования.
9. Перегрузка стандартных операций (унарных, бинарных) в случае, когда оператор-функция принадлежит классу, пример.
10. Перегрузка стандартных операций (унарных, бинарных) в случае, когда оператор-функция не принадлежит классу, пример.
11. Особенности перегрузки операций ++ и – в префиксной и постфиксной формах.
12. Особенности перемещения объекта, продемонстрировать пример перемещения при перегрузке операции «=» (присваивание) на примере своего класса вектор, содержащем в качестве поля динамический массив элементов типа int.
13. Наследование классов. Объявление производного класса, пример использования.
14. Статусы доступа при наследовании классов.
15. Конструкторы и деструктор в производных классах, примеры.
16. Множественное наследование и виртуальные базовые классы, примеры.
17. Переопределение функций при наследовании классов, виртуальные функции в Си++ (таблица виртуальных функций). Статическое и динамическое связывание. Примеры.
18. Абстрактные и локальные классы в Си++, примеры.
19. Требования к классам, объекты которых являются элементами последовательных контейнеров. Лямбда-функции в Си++. Использование лямбда-функций в алгоритмах. Сортировка контейнера алгоритмом sort с помощью предиката, лямбда-функции и перегрузки операции «<».
20. Функциональные объекты (функторы), перегрузка операции () (вызов функции) в классах, пример использования.
21. Требования к классам, объекты которых являются элементами контейнеров set и map.

22. Требования к классам, объекты которых являются элементами контейнеров `unordered_set` и `unordered_map`.
23. Понятие особой (исключительной) ситуации в Си++. Общий формат обработки исключительных ситуаций (ключевые слова: `try`, `catch`, `throw`).
24. Обработка исключения внутри функции, в которой сгенерировано исключение (ключевые слова: `try`, `catch`, `throw`), пример.
25. Обработка исключения вне функции, в которой сгенерировано исключение, объявление функции, в которой генерируются исключения (ключевые слова: `try`, `catch`, `throw`), пример.
26. Создание исключения как объекта класса в Си++ (ключевые слова: `try`, `catch`, `throw`), пример.
27. Формы обработчиков исключений (`catch`), примеры.
28. Формы выражений генерации исключений в Си++ (`throw`), примеры.
29. Обработка исключений в стандартной библиотеке Си++: обзор основных классов.
30. Обработка исключений в стандартной библиотеке Си++: создание своего класса исключения на основе класса `std::exception`, пример.
31. Многозадачность в стандартной библиотеке С++: высокоуровневый интерфейс (функция `std::async()` и шаблон класса `std::future< >`), пример создания потока и получение результата потоковой функции.
32. Многозадачность в стандартной библиотеке С++: низкоуровневый интерфейс (использование класса `std::thread`), пример.
33. Синхронизация в стандартной библиотеке С++. Использование класса `std::mutex` (взаимное исключение), пример.
34. Использование для синхронизации потоков блокировок – шаблона `std::lock_guard`, пример.
35. Использование условных переменных (объекты класса `std::condition_variable`) для синхронизации потоков в стандартной библиотеке С++: посылка и принятие оповещений потоками, примеры.
36. Использование атомарных операций (шаблон `std::atomic`) для синхронизации потоков в стандартной библиотеке С++.
37. Умный указатель (smart pointer) `std::unique_ptr`, внутреннее устройство, пример использования.
38. Умный указатель (smart pointer) `std::shared_ptr`, внутреннее устройство, пример использования.
39. Умный указатель (smart pointer) `std::weak_ptr`, назначение, пример использования.

40. Перегрузка операторов new и delete.
41. Размещающий оператор new и его перегрузка.
42. Передача дополнительных параметров в оператор new при его перегрузке.
43. Дополнительные операции преобразования типов: const_cast, dynamic_cast, static_cast, reinterpret_cast.
44. Понятие о динамической идентификации типов в Си++. Операция typeid. Основные функции класса typeid. Примеры использования.

Типовые задачи:

- 1) Создать класс «целое число».

Элементы класса:

поле задает значение целого числа (статус доступа private);

конструктор для инициализации поля;

метод для вычисления остатка от деления целого числа на 5.

Создать 1 объект данного класса, вывести значение остатка от деления числа на 5.

- 2) Создать класс «целое число».

Элементы класса:

поле задает значение целого числа (статус доступа private);

конструктор для инициализации поля;

метод для печати значения числа в шестнадцатеричной системе счисления.

Создать 1 объект данного класса, вывести на печать значение числа в шестнадцатеричной системе счисления.

- 3) Создать класс «символ».

Элементы класса:

поле задает символ (статус доступа private);

конструктор для инициализации поля;

метод для печати **кода** символа.

Создать 1 объект данного класса, вывести на печать код символ.

- 4) Создать класс «угол (заданный в градусах)».

Элементы класса:

поле задает значение угла в градусах (статус доступа private);

конструктор для инициализации поля;

метод для вычисления синуса угла.

Создать 1 объект данного класса, вывести значение синуса заданного угла.

5) Создать класс «угол (заданный в градусах)».

Элементы класса:

поле задает значение угла в градусах (статус доступа private);

конструктор для инициализации поля;

метод для вычисления косинуса угла.

Создать 1 объект данного класса, вывести значение косинуса заданного угла.

6) Создать класс «целое число».

Элементы класса:

поле задает значение целого числа (статус доступа private);

конструктор для инициализации поля;

метод, возвращающий 1, если число четное, и 0, в противном случае.

Создать 1 объект данного класса, проверить четное ли число, вывести соответствующее сообщение.

7) Создать класс «круг».

Элементы класса:

поле задает значение радиуса круга (статус доступа private);

конструктор для инициализации поля;

метод для вычисления площади круга.

Создать 1 объект данного класса, напечатать значение площади круга.

8) Создать класс «окружность».

Элементы класса:

поле задает значение радиуса окружности (статус доступа private);

конструктор для инициализации поля;

метод для вычисления длины окружности.

Создать 1 объект данного класса, напечатать значение длины окружности.

9) Создать класс «квадрат».

Элементы класса:

поле задает значение длины стороны квадрата (статус доступа private);

конструктор для инициализации поля;

метод для вычисления площади квадрата.

Создать 1 объект данного класса, напечатать значение площади квадрата.

10) Создать класс «квадрат».

Элементы класса:

поле задает значение длины стороны квадрата (статус доступа private);

конструктор для инициализации поля;

метод для вычисления периметра квадрата.

Создать 1 объект данного класса, напечатать значение периметра квадрата.

11) Создать класс базовый класс «точка на плоскости».

Элементы класса:

поля, задающие координаты точки (вещественные поля статус доступа protected);

конструктор для инициализации полей;

метод для печати значений полей.

Создать производный класс «точка в трехмерном пространстве».

Элементы класса:

дополнительное поле, задающее дополнительную координату;

конструктор для инициализации полей;

переопределенный метод для печати значений полей (внутри переопределенного метода в первую очередь должен вызываться метод из базового класса).

Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить динамический полиморфизм.

12) Создать класс базовый класс «вектор на плоскости».

Элементы класса:

поля, задающие координаты точки (вещественные поля статус доступа protected), определяющей конец вектора (начало вектора находится в точке с координатами 0, 0);

конструктор для инициализации полей;

метод для вычисления длины вектора

метод для печати полей и длины вектора.

Создать производный класс «вектор в трехмерном пространстве».

Элементы класса:

дополнительное поле, задающее дополнительную координату;

конструктор для инициализации полей;

переопределенный метод для вычисления длины вектора

переопределенный метод для печати полей и длины вектора (внутри переопределенного метода в первую очередь должен вызываться метод из базового класса).

Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить динамический полиморфизм.

13) Создать класс базовый класс «квадрат».

Элементы класса:

поле, задающее длину стороны;

конструктор для инициализации поля;

метод для вычисления площади квадрата;

метод для печати поля и площади квадрата.

Создать производный класс «куб».

Элементы класса:

конструктор для инициализации поля;

переопределенный метод для вычисления объема куба (вместо площади) (внутри переопределенного метода должен вызываться метод из базового класса).

Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить статический полиморфизм.

14) Создать класс базовый класс «прямоугольник».

Элементы класса:

поля, задающие длины сторон;

конструктор для инициализации полей;

метод для вычисления площади прямоугольника;

метод для печати полей и значения площади.

Создать производный класс «прямоугольный параллелепипед».

Элементы класса:

дополнительное поле, задающее высоту;

конструктор для инициализации полей;

переопределенный метод для вычисления объема (вместо площади) (внутри переопределенного метода должен вызываться метод из базового класса);

переопределенный метод для печати полей и значения объема.

Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить статический полиморфизм.

15) Создать класс базовый класс «круг».

Элементы класса:

поле, задающее радиус;

конструктор для инициализации поля;

метод для вычисления площади круга (площадь круга πr^2);

метод для печати полей и площади.

Создать производный класс «шар».

Элементы класса:

конструктор для инициализации поля;

переопределенный метод для вычисления объема (вместо площади круга) шара (площадь шара $\frac{4}{3}\pi r^3$).

Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового

класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить динамический полиморфизм.

16) Создать класс базовый класс «автомобиль».

Элементы класса:

поле, содержащее наименование модели автомобиля;
поле, содержащее значение максимальной скорости;
конструктор для инициализации полей;
метод для печати параметров автомобиля.

Создать производный класс «грузовой автомобиль».

Элементы класса:

дополнительно поле, содержащее грузоподъемность автомобиля в тоннах;
конструктор для инициализации полей;
переопределенный метод печати параметров автомобиля (внутри переопределенного метода должен вызываться метод из базового класса).
Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить динамический полиморфизм.

17) Создать класс базовый класс «вещественное число».

Элементы класса:

поле, задающее значение числа;
конструктор для инициализации поля;
метод для вычисления модуля числа;
метод для печати поля и модуля числа.

Создать производный класс «комплексное число».

Элементы класса:

дополнительно поле, задающее значение мнимой части числа;
конструктор для инициализации полей;
переопределенный метод для вычисления модуля числа (модуль числа – корень квадратный из суммы квадратов вещественной и мнимой частей числа);
переопределенный метод для печати полей и модуля числа (внутри переопределенного метода должен вызываться метод из базового класса).
Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить динамический полиморфизм.

18) Создать класс базовый класс «сотрудник компании».

Элементы класса:

поле, содержащее ФИО сотрудника;
поле, содержащее размер оклада (статус доступа protected);
конструктор для инициализации полей;
метод для печати ФИО и размера оклада сотрудника.
Создать производный класс «начальник подразделения компании».

Элементы класса:

дополнительное поле, содержащее надбавку к зарплате за исполнение обязанностей начальника;
конструктор для инициализации полей;
переопределенный метод печати ФИО, размера оклада и размера зарплаты начальника (зарплата считается как сумма оклада и надбавки) (внутри переопределенного метода должен вызываться метод из базового класса).
Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить статический полиморфизм.

19) Создать класс базовый класс «вектор на плоскости».

Элементы класса:

поля, задающие координаты точки (вещественные поля статус доступа protected), определяющей конец вектора (начало вектора находится в точке с координатами 0, 0);
конструктор для инициализации полей;
метод для печати координат вектора.
Создать производный класс «вектор в трехмерном пространстве».

Элементы класса:

дополнительное поле, задающее дополнительную координату;
конструктор для инициализации полей;
переопределенный метод для печати координат вектора (внутри переопределенного метода должен вызываться метод из базового класса).
Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить статический полиморфизм.

20) Создать класс базовый класс «квадрат».

Элементы класса:

поле, задающее длину стороны;
конструктор для инициализации поля;
метод для вычисления периметра квадрата;
метод для печати длины стороны и периметра.

Создать производный класс «прямоугольник».

Элементы класса:

дополнительное поле, задающее другую сторону;

конструктор для инициализации полей;

переопределенный метод для вычисления периметра прямоугольника;

переопределенный метод для печати длин сторон и периметра (внутри переопределенного метода должен вызываться метод из базового класса).

Создать по 1 объекту каждого из классов. Объявить переменную - указатель, имеющую тип базового класса, вначале настроить ее на объект базового класса, вызвать метод базового класса через указатель на объект. Затем настроить ее на объект производного класса, вызвать метод производного класса через указатель на объект. Обеспечить статический полиморфизм.

- 21) Реализовать класс «Время». Поля: часы, минуты, секунды. Конструкторы: конструктор для инициализации полей. При недопустимых параметрах объекта создается исключение. Для исключения создать класс, производный от `out_of_range`. Дополнительно созданный класс для исключения содержит поля класса «Время» для недопустимого объекта и метод для печати всех данных. Продемонстрировать работу с объектами в случае возникновения исключения и без возникновения исключения.
- 22) Реализовать класс «Дата». Поля: число, месяц, год. Конструкторы: конструктор для инициализации полей. При недопустимых параметрах объекта создается исключение. Для исключения создать класс, производный от `out_of_range`. Дополнительно созданный класс для исключения содержит поля класса «Дата» для недопустимого объекта и метод для печати всех данных. Продемонстрировать работу с объектами в случае возникновения исключения и без возникновения исключения.
- 23) Разработать функцию для печати n первых значений натуральных чисел (1, 2, 3, ..., n), параметры функции: n и имя потока. Функция печатает имя потока и значение очередного числа на одной строке, далее переход на новую строку. Создать два разных потока для печати двух последовательностей одновременно, обеспечить синхронизацию при печати каждой строки текста (строка печатается потоком полностью, исключить возможность вставки в нее данных другого потока).
- 24) Разработать функцию для печати n значений псевдослучайных чисел (ПСЧ), параметры функции: n и имя потока. Функция печатает имя потока и значение очередного ПСЧ на одной строке, далее переход на новую строку. Создать два разных потока для печати двух последовательностей ПСЧ, обеспечить синхронизацию при печати

каждой строки текста (строка печатается потоком полностью, исключить возможность вставки в нее данных другого потока).

- 25) Разработать функцию для печати n первых значений натуральных чисел (1, 2, 3, ..., n), параметры функции: n и имя потока. Функция печатает имя потока и значение очередного числа на одной строке, далее переход на новую строку. Создать поток, используя эту функцию, при этом главный поток переходит в режим ожидания приема оповещения от дочернего потока. Дочерний поток после завершения печати посылает оповещение главному потоку, главный поток принимает оповещение и печатает сообщение об этом событии. Для отправки и приема оповещений использовать условную переменную (объект класса `std::condition_variable`).
- 26) Разработать функцию для печати n значений псевдослучайных чисел (ПСЧ), параметры функции: n и имя потока. Функция печатает имя потока и значение очередного ПСЧ на одной строке, далее переход на новую строку. Создать поток, используя эту функцию, при этом главный поток переходит в режим ожидания приема оповещения от дочернего потока. Дочерний поток после завершения печати посылает оповещение главному потоку, главный поток принимает оповещение и печатает сообщение об этом событии. Для отправки и приема оповещений использовать условную переменную (объект класса `std::condition_variable`).